

A COOKBOOK FOR NODE.JS DEVELOPERS

Prefect Workflows Cookbook

Orchestrate Python data, ML and AI pipelines with retries, caching, schedules and a UI — framed for a developer who already knows cron, queues and n8n.

Flows

Tasks

Retries

Caching

Concurrency

Deployments

Work pools

Events

Human-in-the-loop

AI pipelines

Written for engineers fluent in Node.js / TypeScript who now run Python jobs and want them observable, retryable and scheduled without building the plumbing.

Prefect 3. Every pattern answers: what it solves, how you would do it with cron or a queue, how Prefect does it, and when NOT to use it. Every snippet was run against a real Prefect server.

Edition 2026 · Prefect 3 · self-hosted server · AI digest capstone

Table of Contents

How to Use This Book	4
The one idea: decorate your functions, get a control plane	4
What this book covers	5
How each chapter is built	5
Setup assumed	6
Part 1 • What Prefect Is (and Where It Fits)	7
1.1 The problem it solves	7
1.2 The mental model	7
1.3 Prefect next to the alternatives	8
1.4 When NOT to use Prefect	8
Part 2 • Install and Your First Flow	9
2.1 Install	9
2.2 A first flow	9
2.3 Start the server and UI	10
2.4 The UI tour	10
2.5 Flows can call flows	10
Part 3 • Tasks in Depth: Retries, Timeouts, Logging	12
3.1 Retries and backoff	12
3.2 Timeouts	13
3.3 Logging	14
3.4 Tags, names and run names	14
3.5 Tasks outside flows	14
Part 4 • Concurrency: submit, map, async	16
4.1 .submit() and futures	16
4.2 .map() for fan-out	16
4.3 Task runners: how submitted tasks execute	17
4.4 Async flows	17
Part 5 • Results and Caching: Don't Do Work Twice	19
5.1 Why caching matters	19
5.2 Cache policies	19
5.3 Where results live	20
5.4 Caching + flow retries = cheap resume	20
Part 6 • States, Hooks and Transactions	21
6.1 Everything has a state	21
6.2 Handle a failure without failing the flow	21
6.3 Hooks: react to the outcome	22
6.4 Transactions: all-or-nothing steps	22
Part 7 • Deployments and Scheduling	24
7.1 The simplest way: flow.serve()	24
7.2 Work pools and workers: run anywhere	24
7.3 prefect.yaml and prefect deploy	25
7.4 Schedules	26
7.5 Run a deployment from code	26
Part 8 • Configuration: Secrets, Variables, Settings	27
8.1 Secrets	27
8.2 Variables	27
8.3 Other blocks	27

8.4 Settings and profiles	28
Part 9 • Events, Automations and Human Input	29
9.1 Everything emits events	29
9.2 Event-triggered deployments	29
9.3 Automations	30
9.4 Pause for a human	30
Part 10 • Testing Flows and Tasks	32
10.1 Test the logic without Prefect: .fn	32
10.2 Test whole flows with a throwaway backend	32
10.3 What to test	33
Part 11 • Production: Self-Hosting, Docker, Limits.	34
11.1 Self-host the server with Postgres	34
11.2 Docker work pools	35
11.3 Concurrency and rate limits	35
11.4 Observability checklist	36
Part 12 • AI and Data Pipelines with Prefect	37
12.1 The LLM task	37
12.2 An embedding and ingestion pipeline (RAG)	38
12.3 A retraining pipeline (MLOps)	38
Part 13 • Capstone: A Scheduled AI Digest	40
13.1 The pipeline	40
13.2 The on-demand trigger	42
13.3 Tests	43
13.4 Run it	43
Part 14 • Cheat Sheet	45
14.1 The skeleton	45
14.2 Doing things concurrently	45
14.3 CLI you'll use	46
14.4 Node.js → Prefect map	46
14.5 The five rules	46
14.6 Prefect vs Temporal vs n8n: pick in 10 seconds	47

How to Use This Book

The one idea: decorate your functions, get a control plane

Prefect turns ordinary Python functions into **observable**, **retryable**, **schedulable** workflows. Put `@flow` on your main function and `@task` on the steps. You then get retries, caching, logs, a run history UI, schedules, parameters and alerts, without rewriting your code as a DAG file or a config language.

PYTHON

```
from prefect import flow, task

@task(retries=3)
def fetch(url: str) -> dict: ...

@flow(log_prints=True)
def daily_report():
    data = fetch("https://api.example.com/stats")
    print(f"got {len(data)} rows")
```

That's a real Prefect workflow. Run it with `python report.py` and it shows up in the UI.

NODE.JS MENTAL MODEL

Think of it as `node-cron` + BullMQ retries + a logging dashboard + a deploy target, for Python functions. You keep writing plain functions. The decorators add the operations layer.

What this book covers

Part	You learn	You can then
1	Where Prefect fits	Choose it over cron, n8n, Airflow or Temporal for the right jobs
2	Install and first flow	Run a flow and read it in the UI
3	Tasks in depth	Retries, backoff, timeouts, logging
4	Concurrency	<code>.submit</code> , <code>.map</code> , async flows, task runners
5	Results and caching	Skip work that's already done
6	States, hooks, transactions	Alert on failure, roll back partial work
7	Deployments and scheduling	Run flows on cron, from the UI or from an API
8	Configuration	Secrets, variables, profiles
9	Events, automations, human input	Event-driven runs and approval steps
10	Testing	Fast tests with a throwaway backend
11	Production	Self-hosting, work pools, Docker, concurrency limits
12	AI and data pipelines	LLM batch jobs with caching and rate limits
13	Capstone	A scheduled, webhook-triggered AI digest pipeline
14	Cheat sheet	One-page recall

How each chapter is built

Every pattern answers: **what it solves**, **how you'd do it with cron or a queue**, **how Prefect does it**, and **when not to use it**. Look for:

NODE.JS MENTAL MODEL

The mapping to Node.js tools you already know.

TIP

A shortcut or habit worth keeping.

COMMON MISTAKE

The trap people fall into.

WHEN NOT TO USE IT

When Prefect, or this feature, is the wrong tool.

EXERCISE

A small build before moving on. The sprint plan tracks these.

Setup assumed

Python functions, decorators and basic `async` (the Python cookbook covers all three), and a terminal. Code targets **Prefect 3** with Python 3.11+ and `uv`. Every snippet in this book was executed against a real Prefect 3 server while it was written.

TIP

Prefect's API moved a lot between versions 1, 2 and 3. Blog posts that mention `Flow(...)` objects, `prefect agent` or `Deployment.build_from_flow` describe older versions. In Prefect 3 you use **workers**, **work pools**, `flow.serve()` and `flow.deploy()`.

Part 1 · What Prefect Is (and Where It Fits)

1.1 The problem it solves

You have Python scripts that matter: a nightly sync, a model retrain, a batch of LLM summaries, a report. Today they probably run from cron on some server. When one fails at 3 a.m. you find out from a customer. When an API flakes, the whole script dies and re-runs everything from scratch. Nobody knows how long it usually takes, or which step broke.

Prefect gives those scripts:

- **Retries** per step, with backoff, so one flaky API call doesn't sink the run.
- **Observability**: every run, every task, its logs, duration and state in a UI.
- **Scheduling and triggers**: cron, intervals, events, API calls or a button.
- **Caching**: don't recompute or re-pay for steps that already succeeded.
- **Parameters**: run the same flow for a different date or customer from the UI.

1.2 The mental model

Prefect	Is	Node.js analogy
Flow	The top-level function; one run = one flow run	Your job's <code>main()</code>
Task	A step inside a flow, with its own retries, cache and state	A BullMQ job step, or a <code>p-retry</code> - wrapped call
State	Scheduled, Pending, Running, Completed, Failed, Crashed, Paused...	A job's status column
Deployment	A flow plus a schedule, parameters and where to run it	A cron entry + a deploy config
Work pool / worker	Where deployment runs execute (process, Docker, Kubernetes)	Your job runners
Server / Cloud	The API + UI that stores runs and schedules	The dashboard and database behind it
Block	Saved config or credentials (<code>Secret</code> , S3 bucket...)	<code>.env</code> values in a secrets manager

NODE.JS MENTAL MODEL

Unlike Temporal, Prefect does **not** replay your code. A flow is a normal Python process. Prefect records what happens and wraps each task with retries and caching. That makes it simpler and very forgiving (you can use any library, any `datetime.now()`), but if the process itself dies mid-run, the run is marked **Crashed**. It doesn't resume at the exact line. You re-run it, and caching skips the steps that already finished.

1.3 Prefect next to the alternatives

	cron + script	n8n	Airflow	Prefect	Temporal
Write it as	Script	Visual canvas	DAG files	Plain Python + decorators	Workflow/activity code
Retries per step	By hand	Per node	Per task	Per task, rich options	Per activity, durable
UI / history	None	Yes	Yes	Yes	Yes
Dynamic workflows (loops, ifs at runtime)	Yes	Limited	Awkward	Natural Python	Natural code
Resume at exact line after a crash	No	No	No	No (cache helps)	Yes
Sweet spot	Tiny jobs	App glue, non-devs	Big data platform teams	Python data / ML / AI pipelines	Business processes, long-running

TIP

Prefect vs Airflow: both orchestrate pipelines. Airflow wants a static DAG declared up front and a heavier platform. Prefect lets the flow **be** Python, with loops, conditionals and dynamic fan-out decided at runtime, and runs locally with zero setup. For a developer coming from Node, Prefect feels far more natural.

1.4 When NOT to use Prefect

WHEN NOT TO USE IT

For a one-off script, just run the script. For glue between SaaS apps built by non-developers, use n8n. For business processes that must survive crashes mid-step and wait for days on human input across deploys (payments, orders, onboarding), Temporal's durable execution is the better fit. Prefect can pause for input, but it isn't built around replay. And for a single background job fired from a web request, a queue (Celery, RQ, BullMQ) is lighter.

EXERCISE

List three scripts or scheduled jobs you run (or would like to run). For each, note what happens today when step 2 of 5 fails, and how you'd find out. Those three become your practice flows.

Part 2 · Install and Your First Flow

2.1 Install

SHELL

```
uv init prefect-lab && cd prefect-lab
uv add prefect httpx
```

2.2 A first flow

PYTHON

```
# report.py
import httpx
from prefect import flow, task

@task(retries=2, retry_delay_seconds=5)
def fetch_users() -> list[dict]:
    r = httpx.get("https://jsonplaceholder.typicode.com/users", timeout=10)
    r.raise_for_status()
    return r.json()

@task
def count_domains(users: list[dict]) -> dict[str, int]:
    counts: dict[str, int] = {}
    for u in users:
        domain = u["email"].split("@")[1]
        counts[domain] = counts.get(domain, 0) + 1
    return counts

@flow(log_prints=True)
def user_report():
    users = fetch_users()
    counts = count_domains(users)
    print(f"{len(users)} users across {len(counts)} domains")
    return counts

if __name__ == "__main__":
    user_report()
```

SHELL

```
uv run report.py
```

With no server configured, Prefect 3 starts a **temporary local API** for the run and records it. Your flow, tasks, logs and states are all tracked, and you changed nothing about how the code runs.

NODE.JS MENTAL MODEL

Note that calling a task inside a flow looks like a normal function call and returns the normal value. There are no promises to await and no `.then`. Prefect intercepts the call, records a task run, applies retries, and hands you the result.

2.3 Start the server and UI

For a UI that remembers runs between executions, run a server and point your shell at it:

SHELL

```
uv run prefect server start                # UI: http://127.0.0.1:4200
uv run prefect config set PREFECT_API_URL=http://127.0.0.1:4200/api
uv run report.py                           # now visible in the UI
```

The local server stores everything in a SQLite file under `~/.prefect`. Part 11 swaps it for Postgres. Alternatively, `prefect cloud login` points you at the hosted Prefect Cloud, which has a free tier.

COMMON MISTAKE

Running the server, but forgetting `PREFECT_API_URL`. Your script silently uses a temporary API instead, and the runs never appear in the UI you have open. `prefect config view` shows which API you're talking to.

2.4 The UI tour

Find these four things. Later chapters refer back to them.

- **Runs:** every flow run with its state, duration and parameters. Click one for the task graph, logs and timeline.
- **Flows:** every flow Prefect has seen, with its run history.
- **Deployments:** flows that can be run on a schedule or on demand (Part 7).
- **Blocks / Variables:** saved configuration and secrets (Part 8).

2.5 Flows can call flows

A flow called inside another flow becomes a **subflow**, with its own run and state, nested under the parent in the UI:

PYTHON

```
@flow
def ingest(day: str) -> int: ...

@flow
def nightly(day: str):
    rows = ingest(day)                # a subflow run
    publish(rows)
```

TIP

Use subflows to group steps that you'd want to see, retry or reuse as one unit (for example "ingest", "train", "evaluate"). Use tasks for individual operations. Everything is still plain Python.

EXERCISE

Run `user_report` three times: once with no server, then with `prefect server start` and `PREFECT_API_URL` set. In the UI, open a run, find the task timeline, and read the `print` output that `log_prints=True` captured.

Part 3 · Tasks in Depth: Retries, Timeouts, Logging

3.1 Retries and backoff

PYTHON

```
from prefect import task
from prefect.tasks import exponential_backoff

@task(retries=3, retry_delay_seconds=10)                # fixed 10s between tries
def fetch_a(): ...

@task(retries=4, retry_delay_seconds=[1, 10, 60, 300])  # explicit schedule
def fetch_b(): ...

@task(retries=5,
      retry_delay_seconds=exponential_backoff(backoff_factor=2),  # 2, 4, 8, 16, 32s
      retry_jitter_factor=0.5)                                     # spread retries out
def fetch_c(): ...
```

Any exception raised by the task triggers a retry, and the UI shows each attempt with its error.

NODE.JS MENTAL MODEL

BullMQ: { attempts: 5, backoff: { type: 'exponential', delay: 2000 } }. p-retry in plain Node. Prefect's version is a decorator argument, and the retry history is visible per run.

To retry only on some errors, pass a predicate:

PYTHON

```
import httpx

def transient(task, task_run, state) -> bool:
    try:
        state.result()
    except httpx.HTTPStatusError as e:
        return e.response.status_code in (429, 500, 502, 503, 504)
    except (httpx.TimeoutException, httpx.ConnectError):
        return True
    return False

@task(retries=5, retry_delay_seconds=exponential_backoff(backoff_factor=1),
      retry_condition_fn=transient)
def call_api(url: str) -> dict:
    r = httpx.get(url, timeout=10)
    r.raise_for_status()
    return r.json()
```

COMMON MISTAKE

Retrying on everything. A `400 Bad Request` or a `KeyError` in your own code fails identically five times, wasting minutes and API quota. Retry only transient failures.

3.2 Timeouts

PYTHON

```
@task(timeout_seconds=30)
def slow_call(): ...

@flow(timeout_seconds=3600)
def whole_job(): ...
```

A timed-out task goes to a failed state (and is retried if it has retries left). Always bound calls to the outside world.

3.3 Logging

PYTHON

```
from prefect import flow, task, get_run_logger

@task
def load(path: str) -> int:
    logger = get_run_logger()
    logger.info("loading %s", path)
    return 42

@flow(log_prints=True)          # also capture plain print() calls
def job():
    n = load("data.csv")
    print(f"loaded {n}")
```

Logs are attached to the run and task in the UI, so there's no more SSH-and-grep.

COMMON MISTAKE

Calling `get_run_logger()` in a helper that you also call outside a flow (for example in a unit test via `my_task.fn()`). Outside a run it raises `MissingContextError`. Part 10 shows `disable_run_logger()` for tests. In plain helpers, use the standard `logging` module.

3.4 Tags, names and run names

PYTHON

```
@task(tags=["llm", "openai"], name="Summarize", task_run_name="summarize-{doc_id}")
def summarize(doc_id: str) -> str: ...

@flow(flow_run_name="report-{day}")
def report(day: str): ...
```

Tags drive filtering in the UI and **tag-based concurrency limits** (Part 11). Templated run names make the runs list readable: `report-2026-09-01` beats `proud-petrel`.

3.5 Tasks outside flows

In Prefect 3, a task can be called on its own, outside any flow, and it's still tracked. That's handy for scripts and notebooks. For anything you'll schedule, wrap tasks in a flow so they share a run and appear together.

WHEN NOT TO USE IT

Don't make every line a task. Each task adds tracking overhead and a row in the UI. A good task is **one meaningful, retryable step**: one API call, one query, one file write, one model call. Pure in-memory transformations can stay as plain functions.

EXERCISE

Write a task that fails with a `503` twice, then succeeds, using a counter. Add `retry_condition_fn` so a `400` fails immediately without retrying. Check both behaviours in the UI.

Part 4 · Concurrency: submit, map, async

By default, calling a task runs it **right away and waits** for the result, one after another. To run tasks concurrently you **submit** them.

4.1 `.submit()` and futures

PYTHON

```
from prefect import flow, task

@task
def fetch(url: str) -> str: ...

@flow
def crawl(urls: list[str]):
    futures = [fetch.submit(u) for u in urls]      # all start now
    pages = [f.result() for f in futures]         # wait for each
    return pages
```

`submit` returns a `PrefectFuture`. `.result()` waits and returns the value, or raises the task's exception.

NODE.JS MENTAL MODEL

`fetch.submit(u)` is `fetch(u)` without the `await`: you get a promise-like future. `[f.result() for f in futures]` is `await Promise.all(promises)`.

4.2 `.map()` for fan-out

PYTHON

```
@flow
def crawl(urls: list[str]):
    futures = fetch.map(urls)                    # one task run per url
    return futures.result()                     # list of results, in order
```

Pass extra fixed arguments with `unmapped`:

PYTHON

```
from prefect import unmapped

futures = score.map(docs, model=unmapped("small-model"))
```

Futures also chain. Passing a future into another task makes Prefect wait for it and records the dependency in the UI's graph:

PYTHON

```
pages = fetch.map(urls)
summaries = summarize.map(pages)           # each waits for its own page
```

4.3 Task runners: how submitted tasks execute

The default runner is a **thread pool**, which is ideal for I/O-bound work like HTTP, databases and LLM calls. Set the pool size per flow:

PYTHON

```
from prefect.task_runners import ThreadPoolTaskRunner

@flow(task_runner=ThreadPoolTaskRunner(max_workers=8))
def crawl(urls: list[str]): ...
```

For CPU-heavy work across machines, the `prefect-dask` and `prefect-ray` integrations provide distributed task runners with the same `.submit` and `.map` code.

4.4 Async flows

If your libraries are async (`httpx.AsyncClient`, `asyncpg`, the async OpenAI client), write async flows and tasks and use `asyncio.gather`:

PYTHON

```
import asyncio
from prefect import flow, task

@task
async def fetch(i: int) -> int:
    await asyncio.sleep(0.1)
    return i

@flow
async def fan_out(n: int = 5) -> int:
    results = await asyncio.gather(*(fetch(i) for i in range(n)))
    return sum(results)

if __name__ == "__main__":
    asyncio.run(fan_out(5))
```

TIP

Pick one style per flow. Async flows with `asyncio.gather` are the natural fit when you're coming from Node. Sync flows with `.submit` and `.map` are the natural fit for sync libraries (`requests` , `pandas` , most SDKs).

COMMON MISTAKE

Submitting 50,000 tasks from one flow. Each task run is tracked by the API, so you'll hammer it and drown the UI. Batch instead: one task handles 500 items, and you map over 100 batches.

EXERCISE

Fetch 20 URLs three ways: sequential calls, `fetch.map(urls)` , and an async flow with `asyncio.gather` . Compare the flow durations in the UI and look at the task timeline for each.

Part 5 · Results and Caching: Don't Do Work Twice

5.1 Why caching matters

A 10-step pipeline fails at step 9. Without caching, the rerun repeats steps 1–8: re-downloading, re-computing and, for LLM work, **re-paying**. With caching, finished steps return their saved results instantly and only step 9 runs again.

This is Prefect's answer to "resume after failure". It isn't Temporal-style replay. It's "skip what's already done".

5.2 Cache policies

PYTHON

```
from datetime import timedelta
from prefect import task
from prefect.cache_policies import INPUTS, TASK_SOURCE

@task(cache_policy=INPUTS + TASK_SOURCE,          # same inputs + same code = cache hit
      cache_expiration=timedelta(days=1),        # store the result so reruns can reuse it
      persist_result=True)
def embed(doc_id: str, text: str) -> list[float]:
    return call_embedding_api(text)
```

Policy	Cache key built from	Use for
INPUTS	Task arguments	Pure functions of their inputs
TASK_SOURCE	The task's source code	Invalidate when you edit the function
INPUTS + TASK_SOURCE	Both	The usual choice
NO_CACHE	—	Side effects that must always run
cache_key_fn=...	Your own function	Custom keys (for example a file's hash)

In a test, calling `embed("d1", "hello")` twice in the same flow computed once. The second call was a cache hit, visible in the UI as a **Cached** state.

NODE.JS MENTAL MODEL

It's memoization (`lodash.memoize` , or a Redis cache in front of a function), except that the cache is shared across runs and machines and it's visible in the UI.

5.3 Where results live

When a result is persisted, Prefect serializes it (pickle by default) and stores it in local storage under `~/.prefect/storage`. For workers on several machines, point result storage at shared storage such as an S3 or GCS bucket block, so a cache written on one worker can be read on another.

SHELL

```
prefect config set PREFECT_RESULTS_PERSIST_BY_DEFAULT=true # persist all task results
```

COMMON MISTAKE

Caching a task with side effects (`send_email` , `charge_card` , `insert_rows`). The second run returns the cached result and **skips the side effect**, which is sometimes exactly right (don't email twice) and sometimes a silent bug (the rows never got inserted into today's table). Decide deliberately, and include the business key (the date, the ID) in the inputs.

COMMON MISTAKE

Returning huge objects (a 2 GB DataFrame) from cached tasks. Every result gets serialized to storage. Write big data to Parquet or S3 yourself and return the path.

5.4 Caching + flow retries = cheap resume

PYTHON

```
@flow(retries=2, retry_delay_seconds=60)
def pipeline(day: str):
    raw = extract(day)           # cached by input
    clean = transform(raw)       # cached by input
    load(clean)                  # the step that flaked
```

If `load` fails, the flow retries after a minute. `extract` and `transform` hit their caches, and only `load` really runs again.

EXERCISE

Add `INPUTS + TASK_SOURCE` caching to an expensive task. Run the flow twice and find the Cached state in the UI. Then edit the task's code, run again, and confirm the cache was invalidated.

Part 6 · States, Hooks and Transactions

6.1 Everything has a state

Every flow run and task run moves through states: **Scheduled** → **Pending** → **Running** → **Completed**, or **Failed** (an exception), **Crashed** (the process died), **Cancelled**, **Paused**, or **Cached**. States drive the UI, retries, alerts and automations.

6.2 Handle a failure without failing the flow

By default, an exception in a task propagates and fails the flow. To inspect a failure and continue, ask for the **state** instead of the value:

PYTHON

```
@flow
def resilient(ids: list[str]):
    done, failed = [], []
    for i in ids:
        s = process(i, return_state=True)
        if s.is_failed():
            failed.append(i)
        else:
            done.append(s.result())
    print(f"{len(done)} ok, {len(failed)} failed: {failed}")
```

`s.result(raise_on_failure=False)` returns the exception object instead of raising it, which is useful for logging.

NODE.JS MENTAL MODEL

It's `Promise.allSettled` for tasks. You get `{status, value | reason}` for each and decide what counts as fatal.

6.3 Hooks: react to the outcome

PYTHON

```
def alert(flow, flow_run, state):
    send_slack(f"{flow.name} run {flow_run.name} ended {state.name}")

@flow(on_failure=[alert], on_crashed=[alert], on_completion=[lambda f, r, s: print("done")])
def nightly(): ...

def task_failed(task, task_run, state):
    print(f"task {task_run.name} failed: {state.message}")

@task(on_failure=[task_failed])
def risky(): ...
```

Hooks run in the same process right after the state change. They're great for Slack or email alerts, cleanup and metrics. For alerts that must fire even when the process is **gone** (a crashed machine), use server-side **automations** (Part 9), since a hook can't run on a dead machine.

6.4 Transactions: all-or-nothing steps

Some steps must be undone if a later step fails. For example, you write a file, the validation of that file fails, and you don't want the bad file left for downstream consumers. Prefect 3 **transactions** group tasks and run **rollback hooks** on failure:

PYTHON

```
from prefect import flow, task
from prefect.transactions import transaction

@task
def write_file(path: str, rows: list[dict]) -> str:
    save_csv(path, rows)
    return path

@write_file.on_rollback
def remove_file(txn):
    os.remove(txn.get("path")) # undo the write

@task
def validate(path: str) -> None:
    if not looks_right(path):
        raise ValueError("bad file")

@flow
def publish(rows: list[dict]):
    with transaction() as txn:
        txn.set("path", "out/report.csv")
        path = write_file("out/report.csv", rows)
        validate(path) # raises -> remove_file runs
```

In testing, the rollback hook ran as soon as `validate` raised, and the flow's `on_failure` hook fired afterwards.

TIP

Transactions are Prefect's lightweight version of the **saga** pattern. For multi-service business sagas that must complete their rollback even if the machine dies halfway, that's the job Temporal was built for.

COMMON MISTAKE

Expecting a rollback to cover side effects in tasks outside the `with transaction()` block, or tasks without an `on_rollback` hook. Only tasks inside the transaction that define a rollback hook get undone.

EXERCISE

Build `publish` with a validation that fails for an empty list. Confirm in the logs that the rollback hook runs and the file is gone. Then add an `on_failure` flow hook that prints a Slack-style message.

Part 7 · Deployments and Scheduling

A **deployment** is a flow plus everything needed to run it without you: a name, default parameters, a schedule or trigger, and where to execute it. Once a flow is deployed you can run it from the UI, the CLI, a schedule, an event or an API call.

7.1 The simplest way: `flow.serve()`

PYTHON

```
# etl.py
from prefect import flow

@flow(log_prints=True)
def etl(day: str = "today"):
    print(f"running ETL for {day}")

if __name__ == "__main__":
    etl.serve(name="nightly", cron="0 2 * * *", parameters={"day": "today"})
```

SHELL

```
uv run etl.py                                # long-running: polls for scheduled
runs
uv run prefect deployment run 'etl/nightly' --param day=2026-09-01  # run now, from
anywhere
```

`serve()` registers the deployment and then **stays running**, executing runs in that process. Use `interval=3600` instead of `cron=...` for "every hour". It's the fastest path from script to scheduled job: one process on any machine.

NODE.JS MENTAL MODEL

It's `node-cron` inside a long-running PM2 process, except that each run is recorded, parameterized, visible in a UI and triggerable remotely.

7.2 Work pools and workers: run anywhere

For production you usually separate **what to run** from **where to run it**. A **work pool** describes an execution environment (a local process, Docker, Kubernetes, a serverless platform), and a **worker** polls the pool and launches each run in that environment.

SHELL

```
uv run prefect work-pool create local-pool --type process
uv run prefect worker start --pool local-pool           # on the machine that runs jobs
```

Deploy a flow to the pool from source code. The worker fetches the code for each run:

PYTHON

```
from prefect import flow

flow.from_source(
    source="https://github.com/you/pipelines.git",    # or a local path while learning
    entrypoint="flows/etl.py:etl",
).deploy(
    name="nightly",
    work_pool_name="local-pool",
    cron="0 2 * * *",
    parameters={"day": "today"},
)
```

For Docker or Kubernetes pools you pass an `image=` instead, and each run starts in a fresh container (Part 11).

Option	Where runs execute	Good for
<code>flow.serve()</code>	Inside the serving process	One box, getting started, simple jobs
Process work pool + worker	Subprocess on the worker's machine	A VM that runs several flows
Docker work pool	New container per run	Isolation, pinned dependencies
Kubernetes work pool	New pod per run	Scale, GPUs, a team platform

COMMON MISTAKE

Creating a deployment and wondering why nothing runs. A work-pool deployment needs a **worker polling that pool**. The run sits in *Scheduled* or *Late* until one starts. With `serve()` the serving process is the worker, so it has to stay running.

7.3 `prefect.yaml` and `prefect deploy`

For many deployments in one repo, declare them in `prefect.yaml` (generate one with `prefect init`) and deploy with `prefect deploy --all`. That file becomes your deploy config under version control, like a `docker-compose.yml` for flows.

7.4 Schedules

Want	Option
Every day at 02:00	<code>cron="0 2 * * *"</code>
Every 15 minutes	<code>interval=900</code>
Business-calendar rules	an RRule schedule (for example "every 2nd Tuesday")
Several schedules on one deployment	<code>schedules=[...]</code>

Schedules have a timezone (set it explicitly, or runs drift around daylight-saving changes) and can be paused from the UI without redeploying.

7.5 Run a deployment from code

PYTHON

```
from prefect.deployments import run_deployment

flow_run = run_deployment(name="etl/nightly", parameters={"day": "2026-09-01"})
print(flow_run.state.name)  # waits for it to finish by default

run_deployment(name="etl/nightly", timeout=0)  # fire and forget
```

This is how your web app, another flow or a webhook handler kicks off a pipeline (Part 13 uses it).

WHEN NOT TO USE IT

Don't create a work pool, Docker images and a Kubernetes cluster for three small nightly jobs. `serve()` on one machine is a legitimate production setup for small teams. Grow into work pools when you need isolation, scale or several machines.

EXERCISE

Serve `etl` with a cron schedule. Trigger it from the CLI with a different `day`, then from Python with `run_deployment`. Then create a process work pool, deploy the same flow to it with `from_source` (a local path is fine), start a worker, and watch the run move from Scheduled to Completed.

Part 8 · Configuration: Secrets, Variables, Settings

8.1 Secrets

PYTHON

```
from prefect.blocks.system import Secret

Secret(value="sk-...").save("openai-api-key", overwrite=True)
# once (or create it in the UI)

@task
def call_llm(prompt: str) -> str:
    key = Secret.load("openai-api-key").get()  # at run time
    ...
```

A `Secret` block is stored by the Prefect server (or Cloud), masked in the UI, and loaded by name in any flow on any worker. There's no `.env` to copy onto each machine.

NODE.JS MENTAL MODEL

It plays the role of `process.env.OPENAI_API_KEY` backed by a secrets manager (AWS Secrets Manager, Doppler, Vault), with a name you reference in code.

8.2 Variables

Non-secret, runtime-tweakable config such as a model name, a batch size or a feature flag:

PYTHON

```
from prefect.variables import Variable

Variable.set("summary_model", "small-model", overwrite=True)
model = Variable.get("summary_model", default="small-model")
```

Change a variable in the UI and the next run picks it up, with no redeploy.

8.3 Other blocks

Blocks are typed, reusable config objects. Integrations add more, for example `prefect-aws` provides `S3Bucket` and `AwsCredentials`, `prefect-slack` provides a Slack webhook block, and

`prefect-sqlalchemy` provides database credentials. Create them in the UI or in code, then `load("name")` them in flows.

8.4 Settings and profiles

Prefect's own settings (`PREFECT_API_URL`, logging level, result persistence...) come from environment variables or **profiles**:

SHELL

```
prefect profile create local && prefect profile use local
prefect config set PREFECT_API_URL=http://127.0.0.1:4200/api
prefect config view                                # what's active, and where it came from
```

Keep one profile per environment (local, staging, cloud) and switch with `prefect profile use`.

COMMON MISTAKE

Hard-coding a secret as a flow **parameter** default. Parameters are stored in plain text on every run and shown in the UI. Secrets go in `Secret` blocks, and parameters hold non-sensitive inputs only.

EXERCISE

Store a fake API key as a `Secret` and a model name as a `Variable`. Read both in a flow, change the variable in the UI, and confirm the next run uses the new value without redeploying.

Part 9 · Events, Automations and Human Input

9.1 Everything emits events

Prefect records an **event** for every state change (`prefect.flow-run.Completed`, `prefect.task-run.Failed`...), and your code can emit its own:

PYTHON

```
from prefect.events import emit_event

emit_event(event="invoice.paid",
           resource={"prefect.resource.id": "invoice.123"},
           payload={"amount": 42})
```

9.2 Event-triggered deployments

Run a flow when an event happens, and pass event data in as parameters:

PYTHON

```
from prefect import flow
from prefect.events import DeploymentEventTrigger

@flow(log_prints=True)
def send_receipt(invoice_id: str = "none"):
    print("sending receipt for", invoice_id)

if __name__ == "__main__":
    send_receipt.serve(
        name="on-invoice-paid",
        triggers=[DeploymentEventTrigger(
            expect={"invoice.paid"},
            parameters={"invoice_id": "{{ event.resource.id }}"},
        )],
    )
```

In testing, emitting `invoice.paid` for `invoice.999` started a run with `invoice_id="invoice.999"` a few seconds later.

NODE.JS MENTAL MODEL

It's an event bus (Redis pub/sub, or EventBridge) wired straight to "run this job with these params", with the wiring declared next to the flow.

9.3 Automations

Automations are server-side rules: *when* something happens (or fails to happen), *do* something. You create them in the UI (or via the API). Common ones:

- A flow run fails or crashes → send a Slack or email notification.
- A deployment hasn't completed in 26 hours → alert (it's a "proactive" trigger, which fires on **absence** of an event).
- A run has been Running longer than 2 hours → cancel it and notify.
- A custom event arrives → run a deployment, pause a schedule, or call a webhook.

Because automations run on the server, they still fire when the machine running your flow has died, which in-process hooks (Part 6.3) can't do.

9.4 Pause for a human

A flow can stop, ask for typed input in the UI, and continue with the answer:

PYTHON

```
from prefect import flow
from prefect.flow_runs import pause_flow_run
from prefect.input import RunInput

class Approval(RunInput):
    approved: bool
    note: str = ""

@flow(log_prints=True)
def publish_report(report_id: str):
    draft = build_draft(report_id)
    answer = pause_flow_run(wait_for_input=Approval, timeout=3600) # UI shows a form
    if answer.approved:
        publish(draft)
    print("reviewer said:", answer.note)
```

The run goes to **Paused**, the UI renders a form from the `Approval` model, and the reviewer's answer arrives as a typed object. You can also resume from code with `resume_flow_run(flow_run_id, run_input={...})`, which is how a Slack button handler would do it.

TIP

`pause_flow_run` keeps the flow process alive while it waits, which is fine for minutes or hours. For waits of days, `suspend_flow_run` releases the infrastructure and re-runs the flow on resume. Cache the earlier tasks so the rerun skips them. If multi-day waits are the core of the process, that's a strong sign for Temporal.

EXERCISE

Build `publish_report` with a pause. Resume it once from the UI form, and once from a Python script with `resume_flow_run`. Then create a UI automation that notifies you when any run of this flow fails.

Part 10 · Testing Flows and Tasks

10.1 Test the logic without Prefect: `.fn`

Every task and flow keeps the original function at `.fn`. Unit-test the logic directly:

PYTHON

```
from report import count_domains

def test_count_domains():
    users = [{"email": "a@x.com"}, {"email": "b@x.com"}, {"email": "c@y.com"}]
    assert count_domains.fn(users) == {"x.com": 2, "y.com": 1}
```

If the task uses `get_run_logger()`, wrap the call so there's no missing-context error:

PYTHON

```
from prefect.logging import disable_run_logger

def test_load():
    with disable_run_logger():
        assert load.fn("data.csv") == 42
```

10.2 Test whole flows with a throwaway backend

`prefect_test_harness` spins up a temporary, isolated Prefect database for the test session, so tests never touch your real server:

PYTHON

```
# conftest.py
import pytest
from prefect.testing.utilities import prefect_test_harness

@pytest.fixture(autouse=True, scope="session")
def prefect_db():
    with prefect_test_harness():
        yield
```

PYTHON

```
# test_flows.py
import asyncio
from flows import fan_out, user_report

def test_fan_out():
    assert asyncio.run(fan_out(5)) == 10

def test_user_report(monkeypatch):
    monkeypatch.setattr("flows.fetch_users.fn", lambda: [{"email": "a@x.com"}])
    assert user_report() == {"x.com": 1}
```

NODE.JS MENTAL MODEL

It's Jest with an in-memory database: `beforeAll` spins it up, tests run real flows against it, and it's thrown away after. `.fn` is like importing the un-wrapped function for a pure unit test.

COMMON MISTAKE

Starting the harness per test function. It takes several seconds to boot, so use a **session-scoped** fixture as above.

10.3 What to test

Level	How	Covers
Task logic	<code>task.fn(...)</code>	Transformations, parsing, business rules
Flow wiring	Call the flow under the harness, with I/O tasks mocked	Branching, parameters, return values
Integration	A real run against a staging server	Blocks, deployments, infrastructure

EXERCISE

Add a session-scoped harness fixture, a `.fn` unit test for one task, and a flow test that mocks the HTTP task. Run `pytest` and confirm nothing appears on your real server's UI.

Part 11 · Production: Self-Hosting, Docker, Limits

11.1 Self-host the server with Postgres

The default SQLite database is fine for one person. For a team, or anything important, run the server on Postgres:

YAML

```
# docker-compose.yml
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: prefect
      POSTGRES_PASSWORD: prefect
      POSTGRES_DB: prefect
    volumes: [pgdata:/var/lib/postgresql/data]

  prefect:
    image: prefecthq/prefect:3-latest
    command: prefect server start --host 0.0.0.0
    environment:
      PREFECT_API_DATABASE_CONNECTION_URL: postgresql+asyncpg://prefect:prefect@db:5432/
    ports: ["4200:4200"]
    depends_on: [db]

volumes:
  pgdata:
```

SHELL

```
docker compose up -d
prefect config set PREFECT_API_URL=http://localhost:4200/api
```

TIP

Pin the image to a specific Prefect version in production (`prefecthq/prefect:3.x.y-python3.12`) and keep your workers' `prefect` package on the same version. `3-latest` is for learning.

COMMON MISTAKE

Exposing a self-hosted Prefect server to the internet with no auth in front of it. The open-source server has only basic auth options. Keep it on a private network or VPN, or behind a reverse proxy with authentication. Prefect Cloud adds users, roles and SSO if you need them.

11.2 Docker work pools

Run each flow run in a fresh container with pinned dependencies:

SHELL

```
uv add prefect-docker
prefect work-pool create docker-pool --type docker
prefect worker start --pool docker-pool
```

PYTHON

```
etl.deploy(
    name="nightly",
    work_pool_name="docker-pool",
    image="registry.example.com/pipelines:2026-09-01",    # built and pushed by CI
    cron="0 2 * * *",
    build=False, push=False,                            # CI already built it
)
```

The same pattern works for Kubernetes (`prefect-kubernetes`, `--type kubernetes`) with a pod per run, which is how you get GPU jobs and autoscaling (see the Cloud and Kubernetes cookbook).

11.3 Concurrency and rate limits

Protect APIs and databases from your own fan-out.

Tag-based task limits: at most 5 `llm`-tagged tasks run at once, across all flows:

SHELL

```
prefect concurrency-limit create llm 5
```

PYTHON

```
@task(tags=["llm"])
def summarize(text: str) -> str: ...
```

Global concurrency and rate limits in code:

SHELL

```
prefect gcl create openai --limit 5 # 5 at a time
prefect gcl create openai-rpm --limit 60 --slot-decay-per-second 1 # ~60 per minute
```

PYTHON

```
from prefect.concurrency.sync import concurrency, rate_limit

@task
def call_llm(prompt: str) -> str:
    rate_limit("openai-rpm") # blocks until a slot frees up
    with concurrency("openai", occupy=1): # at most 5 in flight
        return client.complete(prompt)
```

NODE.JS MENTAL MODEL

It's p-limit and a token-bucket rate limiter (bottleneck), except they're enforced by the server, so the limits hold across every worker and every flow, not just inside one process.

11.4 Observability checklist

- **Artifacts:** publish a Markdown summary or a table per run (`create_markdown_artifact`, `create_table_artifact`). They show up on the run page, which works well as a daily report.
- **Automations** for failure, crash and "didn't run" alerts (Part 9.3).
- **Run names** that include the business key (Part 3.4).
- **Logs** via `get_run_logger` or `log_prints=True`, searchable in the UI.
- **Metrics:** the server exposes run states you can export to your monitoring stack. Prefect Cloud adds dashboards.

WHEN NOT TO USE IT

Don't self-host if nobody on the team wants to own a Postgres-backed service. Prefect Cloud's free tier covers small workloads, and your code and workers stay on your infrastructure either way.

EXERCISE

Run the compose file above, point your profile at it, and deploy one flow to a Docker work pool. Add a `concurrency("openai", occupy=1)` block with a limit of 2, map 10 tasks, and watch in the timeline that only two run at a time.

Part 12 · AI and Data Pipelines with Prefect

Prefect's sweet spot is **batch** work: embed 10,000 documents, summarize yesterday's tickets, re-score leads, retrain a model. These are Python-heavy, parallel, flaky (APIs rate-limit) and expensive, which is exactly what retries, caching and concurrency limits address.

12.1 The LLM task

PYTHON

```
from prefect import task
from prefect.blocks.system import Secret
from prefect.cache_policies import INPUTS, TASK_SOURCE
from prefect.concurrency.sync import concurrency
from prefect.tasks import exponential_backoff

@task(retries=4, retry_delay_seconds=exponential_backoff(backoff_factor=2), retry_jitter_factor=0.5,
      cache_policy=INPUTS + TASK_SOURCE, persist_result=True,
      tags=["llm"], task_run_name="summarize-{doc_id}")
def summarize(doc_id: str, text: str, model: str) -> str:
    from openai import OpenAI
    client = OpenAI(api_key=Secret.load("openai-api-key").get())
    with concurrency("openai", occupy=1):
        resp = client.chat.completions.create(
            model=model,
            messages=[{"role": "system", "content": "Summarize in 3 bullet points."},
                      {"role": "user", "content": text}])
    return resp.choices[0].message.content
```

Every property is doing work here:

- **Retries with jitter** absorb 429s and 5xx errors.
- The **cache** keyed on inputs and source means a rerun never pays twice for the same document and prompt.
- The **concurrency limit** keeps you under the provider's limits across all flows.
- The **run name** makes each call findable in the UI.

12.2 An embedding and ingestion pipeline (RAG)

PYTHON

```
@flow(log_prints=True)
def ingest(folder: str, batch_size: int = 100):
    docs = list_documents(folder) # task: returns paths
    batches = [docs[i:i + batch_size] for i in range(0, len(docs), batch_size)]
    chunked = chunk_batch.map(batches) # task: split into chunks
    vectors = embed_batch.map(chunked) # task: cached, rate-
    limited
    upsert_batch.map(vectors) # task: write to pgvector
    print(f"ingested {len(docs)} docs in {len(batches)} batches")
```

This is the ingestion side of the RAG cookbook as a scheduled, observable pipeline. Batching keeps the number of task runs manageable, and caching makes re-ingesting an unchanged folder nearly free.

12.3 A retraining pipeline (MLOps)

PYTHON

```
@flow
def retrain(min_auc: float = 0.80):
    data = extract_training_data()
    model, metrics = train(data)
    if metrics["auc"] < min_auc:
        raise ValueError(f"AUC {metrics['auc']:.3f} below {min_auc}") # fails loudly,
    triggers alert
    register_model(model, metrics) # for example the
    MLflow registry
    create_markdown_artifact(key="retrain-report",
                             markdown=f"# Retrain\nAUC: **{metrics['auc']:.3f}**")
```

Schedule it weekly, and add an automation that alerts on failure. That's the "continuous training" loop from the MLOps cookbook with very little code.

COMMON MISTAKE

Putting the whole batch of 10,000 documents in **one** task. One failure retries all 10,000 calls, and the cache can't help. Make the unit of retry and caching **one document or one small batch**.

WHEN NOT TO USE IT

For an interactive chatbot or agent answering a user in real time, Prefect is the wrong layer. That's request/response code in your API (the AI Agents cookbook). For long-running agents that wait on humans for days, Temporal fits better. Prefect is for **batch and scheduled** AI work.

EXERCISE

Build `summarize` with a fake LLM that fails 30% of the time. Map it over 50 documents with a concurrency limit of 3. Run the flow twice and confirm the second run is almost entirely **Cached** and makes no new "LLM calls".

Part 13 · Capstone: A Scheduled AI Digest

You'll put the whole book together in one pipeline you might actually keep: every morning it fetches the top Hacker News stories, removes duplicates, summarizes each one with an LLM (retried, cached and rate-limited), publishes a Markdown report on the run page, and posts it to Slack. It runs on a schedule **and** on demand through an HTTP endpoint, n8n-style.

n8n concept	This pipeline in Prefect
Schedule Trigger node	<code>daily_digest.serve(cron="0 7 * * *")</code>
Webhook Trigger node	FastAPI POST /digest → <code>run_deployment(...)</code>
HTTP Request node + "Retry on fail"	<code>fetch_top_stories</code> task with <code>retries</code> and backoff
Code node	Plain Python (<code>dedupe</code>)
OpenAI node	<code>summarize</code> task: cached, rate-limited, retried
Credentials	<code>Secret blocks (openai-api-key , slack-webhook-url)</code>
Executions list	Prefect runs UI + the <code>daily-digest</code> artifact

13.1 The pipeline

PYTHON

```
# digest.py – a scheduled, rate-limited, cached AI news digest
import hashlib
from datetime import date

import httpx
from prefect import flow, task, get_run_logger
from prefect.artifacts import create_markdown_artifact
from prefect.blocks.system import Secret
from prefect.cache_policies import INPUTS, TASK_SOURCE
from prefect.concurrency.sync import concurrency
from prefect.tasks import exponential_backoff
from prefect.variables import Variable

HN = "https://hacker-news.firebaseio.com/v0"

@task(retries=3, retry_delay_seconds=exponential_backoff(backoff_factor=2), task_run_name="f
etch-top-{{limit}}")
def fetch_top_stories(limit: int = 15) -> list[dict]:
    with httpx.Client(timeout=15) as http:
        ids = http.get(f"{HN}/topstories.json").raise_for_status().json()[0:limit]
        items = [http.get(f"{HN}/item/{i}.json").raise_for_status().json() for i in ids]
    return [{"id": str(it["id"]), "title": it.get("title", ""), "url": it.get("url", "")},
```

```

        "text": it.get("text", "") or it.get("title", "")} for it in items if it]

def dedupe(items: list[dict]) -> list[dict]:          # plain function: pure, fast, no task
    needed
    seen, out = set(), []
    for it in items:
        key = hashlib.sha1(it["title"].lower().encode()).hexdigest()
        if key not in seen:
            seen.add(key)
            out.append(it)
    return out

def llm_complete(prompt: str, model: str) -> str:
    try:
        key = Secret.load("openai-api-key").get()
    except ValueError:                                # no secret saved: offline fallback
        return f"- {prompt.splitlines()[-1][:90]}"
    from openai import OpenAI
    resp = OpenAI(api_key=key).chat.completions.create(
        model=model, messages=[{"role": "user", "content": prompt}])
    return resp.choices[0].message.content

@task(retries=4, retry_delay_seconds=exponential_backoff(backoff_factor=2), retry_jitter_factor=0.5,
      cache_policy=INPUTS + TASK_SOURCE, persist_result=True,
      tags=["llm"], task_run_name="summarize-{item_id}")
def summarize(item_id: str, title: str, text: str, model: str) -> str:
    with concurrency("llm", occupy=1, strict=False):
        return llm_complete(f"One-line takeaway for a backend engineer:\n{title}\n{text[:2000]}", model)

@task
def publish(day: str, rows: list[tuple[dict, str]]) -> str:
    md = [f"# AI digest · {day}", ""]
    md += [f"**[{it['title']}]({it['url'] or '#'})**\n\n{summary}\n" for it, summary in rows]
    body = "\n".join(md)
    create_markdown_artifact(key="daily-digest", markdown=body, description=f"Digest for {day}")
    return body

@task(retries=2, retry_delay_seconds=10)
def notify(text: str) -> None:
    try:
        url = Secret.load("slack-webhook-url").get()
    except ValueError:
        get_run_logger().info("no slack-webhook-url secret; skipping notify")
        return
    httpx.post(url, json={"text": text[:3000]}, timeout=10).raise_for_status()

@flow(log_prints=True, flow_run_name="digest-{day}")
def daily_digest(day: str = "", limit: int = 15) -> int:
    day = day or date.today().isoformat()

```

```

model = Variable.get("digest_model", default="gpt-4o-mini")
items = dedupe(fetch_top_stories(limit))
futures = [summarize.submit(it["id"], it["title"], it["text"], model) for it in items]
rows = list(zip(items, [f.result() for f in futures]))
body = publish(day, rows)
notify(body)
print(f"digest for {day}: {len(rows)} stories")
return len(rows)

if __name__ == "__main__":
    daily_digest.serve(name="morning", cron="0 7 * * *")

```

Design notes:

- `dedupe` is a **plain function**, not a task. It's pure, instant and not worth tracking.
- `summarize` takes the story's **ID and text as inputs**, so the cache key changes when the story changes, and a rerun on the same morning costs nothing.
- `concurrency("llm", ..., strict=False)` respects a global limit if you created one (`prefect gcl create llm --limit 3`) and simply proceeds if you haven't.
- Without saved secrets, the flow runs **offline**: a fake one-line summary and no Slack post. You can build and test it before touching any API key.

13.2 The on-demand trigger

PYTHON

```

# api.py - trigger the digest on demand (a "webhook trigger", n8n-style)
from fastapi import FastAPI
from prefect.deployments import run_deployment

app = FastAPI()

@app.post("/digest")
def trigger(limit: int = 15):
    run = run_deployment(name="daily-digest/morning", parameters={"limit": limit},
        timeout=0)
    return {"flow_run_id": str(run.id), "state": run.state.name if run.state else None}

```

13.3 Tests

PYTHON

```
import pytest
from prefect.testing.utilities import prefect_test_harness
import digest

FAKE = [{"id": "1", "title": "Temporal 2", "url": "u1", "text": "durable"},
        {"id": "2", "title": "temporal 2", "url": "u2", "text": "dup"},
        {"id": "3", "title": "Prefect tips", "url": "u3", "text": "flows"}]

@pytest.fixture(autouse=True, scope="session")
def prefect_db():
    with prefect_test_harness():
        yield

def test_dedupe():
    assert [i["id"] for i in digest.dedupe(FAKE)] == ["1", "3"]

def test_flow(monkeypatch):
    monkeypatch.setattr("digest.fetch_top_stories.fn", lambda limit=15: FAKE)
    assert digest.daily_digest(day="2026-09-01") == 2
```

13.4 Run it

SHELL

```
uv add prefect httpx fastapi uvicorn openai
uv run prefect server start # terminal 1
uv run prefect config set PREFECT_API_URL=http://127.0.0.1:4200/api
uv run python -c "from prefect.blocks.system import Secret;
Secret(value='sk-...').save('openai-api-key')"
uv run prefect gcl create llm --limit 3
uv run digest.py # terminal 2: serves the
schedule
uv run uvicorn api:app --port 8000 # terminal 3
curl -X POST 'localhost:8000/digest?limit=5' # run now
```

Open the run in the UI: the task graph shows `fetch-top-5`, then five `summarize-<id>` runs (never more than three at once), then `publish` and `notify`. The **Artifacts** tab holds today's digest. Trigger it again and the summaries come back **Cached**.

TIP

This is the same automation you might build in n8n in 15 minutes. The Prefect version costs more upfront and gives you tests, version control, typed code, caching that saves real money, and a pipeline that grows into RAG ingestion or retraining without changing tools.

EXERCISE

Extend the digest: (1) add a second source (an RSS feed via `feedparser`) and merge before dedupe ; (2) add a `Variable` for the story limit; (3) add an automation that alerts you if the `morning` deployment hasn't completed by 08:00; (4) add a `pause_flow_run` approval before `notify` on weekends.

Part 14 · Cheat Sheet

14.1 The skeleton

PYTHON

```
from datetime import timedelta
from prefect import flow, task, get_run_logger
from prefect.cache_policies import INPUTS, TASK_SOURCE
from prefect.tasks import exponential_backoff

@task(retries=3, retry_delay_seconds=exponential_backoff(backoff_factor=2),
      timeout_seconds=60, cache_policy=INPUTS + TASK_SOURCE,
      cache_expiration=timedelta(days=1), persist_result=True, tags=["api"])
def step(x: str) -> str: ...

@flow(log_prints=True, retries=1, flow_run_name="job-{day}")
def job(day: str):
    results = step.map(["a", "b", "c"]).result()
    print(results)

if __name__ == "__main__":
    job.serve(name="daily", cron="0 6 * * *")
```

14.2 Doing things concurrently

Want	Write
Run now, wait for the value	<code>step(x)</code>
Start without waiting	<code>f = step.submit(x) then f.result()</code>
One run per item	<code>step.map(items).result()</code>
Fixed extra argument	<code>step.map(items, model=unmapped("m"))</code>
Async fan-out	<code>await asyncio.gather(*(step(i) for i in items))</code> in an async flow
Inspect failure, continue	<code>s = step(x, return_state=True); s.is_failed()</code>

14.3 CLI you'll use

SHELL

```

prefect server start                                # UI :4200
prefect config set PREFECT_API_URL=http://127.0.0.1:4200/api
prefect config view
prefect deployment ls
prefect deployment run 'flow-name/deployment-name' --param day=2026-09-01
prefect work-pool create my-pool --type process      # or docker / kubernetes
prefect worker start --pool my-pool
prefect gcl create openai --limit 5                 # global concurrency limit
prefect concurrency-limit create llm 5              # tag-based task limit
prefect variable ls

```

14.4 Node.js → Prefect map

In Node you'd use	Prefect gives you
<code>node-cron</code> in a PM2 process	<code>flow.serve(cron=...)</code>
BullMQ attempts + backoff	<code>@task(retries=..., retry_delay_seconds=...)</code>
<code>p-limit</code> / bottleneck	<code>concurrency(...), rate_limit(...)</code> , tag limits
<code>lodash.memoize</code> / Redis cache	<code>cache_policy=INPUTS + TASK_SOURCE</code>
<code>Promise.all</code> / <code>allSettled</code>	<code>.map()</code> / <code>return_state=True</code>
<code>.env</code> + a secrets manager	Secret blocks + Variables
Winston logs + grep	<code>get_run_logger()</code> + the runs UI
Hand-built status dashboard	The Prefect UI + artifacts

14.5 The five rules

1. **A task is one retryable, meaningful step.** Not every line, and not the whole job.
2. **Retry only transient failures.** Use `retry_condition_fn` for anything with 4xx errors.
3. **Cache anything expensive and deterministic**, with the business key in the inputs.
4. **Secrets in blocks, never in parameters.**
5. **A deployment needs something polling it:** a `serve()` process or a worker.

14.6 Prefect vs Temporal vs n8n: pick in 10 seconds

If the job is...	Pick
A Python data, ML or AI pipeline you schedule and monitor	Prefect
A business process that must never lose its place, or waits days for people	Temporal
Glue between SaaS apps, built fast, maybe by non-developers	n8n
One background job from a web request	A queue (Celery, RQ, BullMQ)